🧪 QA Daily Log — DevOps Documentation

Complete overview of architecture, deployment, tests, and system management

Vibecoded by Bosko

📅 March 2026 | 👤 Bosko Sretenovic | 🏷️ v2.0 — Raspberry Pi + Cloudflare Tunnel

1. URLS AND ACCESS

SERVICE	URL / ADDRESS	NOTES
App (users)	`https://qa-daily-log.online`	Public URL — Cloudflare + Tunnel
Health check	`https://qa-daily-log.online/api/health`	Returns `{ ok: true }`
GitHub repo	`https://github.com/sretenovicue/qa-daily-log`	Source code
Neon DB console	`https://console.neon.tech`	PostgreSQL dashboard
Cloudflare console	`https://dash.cloudflare.com`	DNS + Tunnel management
Namecheap	`https://namecheap.com`	Domain registrar (renew Mar 2027)
Raspberry Pi (SSH)	`ssh sretenovicue@nasty.local`	Local network, port 22

2. SYSTEM ARCHITECTURE (V2.0)

```
┌──────────────────────────────────────────────────────────────────────────────┐ BROWSER
│ https://qa-daily-log.online │
└──────────────────────────────┬───────────────────────────────────────────────┘ HTTPS
port 443 ▼ ┌──────────────────────────────────────────────────────────────────┐
CLOUDFLARE EDGE (free plan) │ • SSL termination (HTTPS ↔ Tunnel) │ • DDoS
protection — Pi's real IP is NEVER exposed to the internet │ • DNS proxy (CNAME
Tunnel record) │
└──────────────────────────────┬───────────────────────────────────────────────┘ Cloudflare
Tunnel (TLS, outbound connection — ISP port blocking irrelevant) ▼
┌──────────────────────────────────────────────────────────────────────────────┐ RASPBERRY
PI 4B (hostname: nasty, Debian Linux) │ │ cloudflared (systemd) ──────→
http://localhost:3001 │ │ PM2 → node server.js (port 3001) │ │ ├──
/api/* → Express routes │ │ └── /* → React dist/ ──┘ │
└──────────────────────────────┬───────────────────────────────────────────────┘ PostgreSQL
SSL (Neon cloud, Frankfurt) ▼
┌──────────────────────────────────────────────────────────────────────────────┐ NEON
POSTGRESQL (serverless, free tier) │ ep-jolly-king-a9bnwjuq — gwc.azure.neon.tech │
└──────────────────────────────────────────────────────────────────────────────┘
```

Key advantage of Cloudflare Tunnel: The Pi requires no open inbound ports. Cloudflared establishes an *outbound* connection to Cloudflare — ISP blocking of inbound ports (80, 443) has no effect. The Pi's real IP remains hidden.

REQUEST FLOW:

```
Browser → Cloudflare edge → Cloudflare Tunnel → cloudflared (Pi) →
Express :3001 → Neon DB
```

3. RASPBERRY PI — HARDWARE AND OS

HARDWARE

Model	Raspberry Pi 4B
Hostname	`nasty`
OS	Debian Linux (Raspberry Pi OS)
User	`sretenovicue`
SSH access	`ssh sretenovicue@nasty.local`

INSTALLED SERVICES

Node.js	v20.20.1
npm	v10.8.2
PM2	Process manager (auto-start on boot)
cloudflared	Cloudflare Tunnel daemon (systemd)
OpenMediaVault	NAS UI (separate port, no conflict)

APP LOCATION ON PI:

```
/home/sretenovicue/qa-daily-log/     ← repository root (git clone)
├── backend/
│   ├── server.js                    ← Express (also serves React dist/)
│   ├── db.js
│   ├── .env                         ← production env variables (NOT in git)
│   └── routes/
└── frontend/
    ├── src/
    └── dist/                        ← React build (served by Express)
```

4. CLOUDFLARE TUNNEL

TUNNEL CONFIGURATION

Tunnel ID	`2db48967-9405-40c3-b595-07fc3912589c`
Tunnel name	`qa-daily-log`
Hostname	`qa-daily-log.online`
Target service	`http://localhost:3001`
Service type	systemd — `cloudflared.service`
Auto-start	Starts automatically on Pi reboot

FILES ON PI

Config	`/etc/cloudflared/config.yml`
Credentials	`/etc/cloudflared/2db48967...json`
Cert	`/etc/cloudflared/cert.pem`

DNS RECORD (CLOUDFLARE)

Type	CNAME (Tunnel)
Name	`qa-daily-log.online`
Proxy	Proxied 🟠

CONFIG.YML:

```
tunnel: 2db48967-9405-40c3-b595-07fc3912589c
credentials-file: /etc/cloudflared/2db48967-9405-40c3-b595-07fc3912589c.json

ingress:
  - hostname: qa-daily-log.online
    service: http://localhost:3001
  - service: http_status:404
```

SERVICE MANAGEMENT:

```
sudo systemctl status cloudflared      # check status
sudo systemctl restart cloudflared     # restart
sudo journalctl -u cloudflared -f      # live logs
```

5. DOMAIN AND DNS

DOMAIN

Domain	`qa-daily-log.online`
Registrar	Namecheap
Price	$0.98 / yr (renewal Mar 2027, ~$2.88/yr)
Nameservers	Cloudflare (mcgrory + rayne)
SSL	Cloudflare auto — free HTTPS

DNS FLOW

1. User opens `qa-daily-log.online`
2. DNS resolves to Cloudflare nameservers
3. Cloudflare proxy receives the request
4. Cloudflare forwards request through Tunnel
5. cloudflared on Pi receives and forwards to `:3001`
6. Express responds with HTML or JSON

6. BACKEND — NODE.JS + PM2

PM2 CONFIGURATION

Process name	`qa-backend`
Start command	`node server.js`
Working dir	`~/qa-daily-log/backend`
Port	`3001`
Auto-start	`pm2 startup` (systemd)

BACKEND/.ENV (PRODUCTION)

`DATABASE_URL`	Neon connection string (SSL)
`PORT`	`3001`
`ALLOWED_ORIGIN`	`https://qa-daily-log.online`
`JWT_SECRET`	Secret key (min 32 chars)

Frontend serving: Express serves the React build from `frontend/dist/`. No separate web server needed — one Node.js process handles everything. The `GET *` route returns `index.html` for SPA routing.

PM2 COMMANDS:

```
pm2 status                  # list processes (status, CPU, RAM)
pm2 logs qa-backend         # live logs
pm2 restart qa-backend      # restart
pm2 monit                   # real-time monitor
pm2 save                    # save process list for auto-start
```

7. DEPLOYMENT — PUBLISHING A NEW VERSION

STEP 1 — LOCAL MACHINE (PUSH CODE):

```
cd ~/Desktop/testApp
git add .
git commit -m "description of changes"
git push origin main
```

STEP 2 — RASPBERRY PI (SSH):

```
ssh sretenovicue@nasty.local
cd ~/qa-daily-log

git pull origin main                          # pull new changes

# If there are new npm packages:
cd backend  && npm install && cd ..
cd frontend && npm install && cd ..

cd frontend && npm run build && cd ..         # rebuild React app

pm2 restart qa-backend                        # restart backend
```

`git push` → `ssh to Pi` → `git pull` → `npm run build` → `pm2 restart ✓`

⚠️ **Important:** Always run `npm run build` in the `frontend/` folder after React code changes. Express serves the static `dist/` folder — without a rebuild, users see the old version.

8. APP CHANGES — CHANGELOG

FEATURE	DESCRIPTION	FILES
i18n SR / EN	Bilingual support — Serbian and English. Flag switcher (🇷🇸 /🇬🇧) in header. Language persisted in localStorage. Date format localised (`sr-Latn` / `en-GB`).	`i18n.js` , `locales/sr.json` , `locales/en.json`
Avatar upload	Users can upload a profile picture by clicking the avatar in the header. Image is resized to max 200×200px, compressed as JPEG, stored as base64 in the database.	`App.jsx` , `Avatar.jsx` , `routes/users.js`
User title	Manager can assign a title to any user inline (click in the table). Title is displayed below the username in the header.	`UsersPanel.jsx` , `App.jsx`
Guest role	Special role for demo access — can view everything but cannot add entries. Demo button on the login page (no password required).	`AuthPage.jsx` , `App.jsx` , `middleware/auth.js`
Weekly Report tab	Weekly activity report for managers and guests — overview of activity by week.	`WeeklyReport.jsx`
Easter eggs — per user	Personalised animations and quotes for specific users: Sopranos quotes, Stranger Things quotes, Taxi Driver quotes, Đoković wink, fish animation (Stanko), skier animation (Jana).	`QuoteToast.jsx` , `DjokovicWink.jsx` , `FishAnimation.jsx` , `SkierAnimation.jsx`
Pending approval badge	Red badge on the Users tab showing the number of users awaiting approval. Auto-refreshes every 60 seconds.	`App.jsx` , `store.js`
Raspberry Pi deploy	Migrated from Netlify+Render to self-hosted Pi. Express serves both frontend and backend. Cloudflare Tunnel for public access without open inbound ports.	`server.js` , `backend/.env`

9. UNIT TESTS — VITEST

RUNNING TESTS

```
cd frontend
npm test            # run once
npm run test:watch # watch mode
```

TEST FILES

`validate.test.js`	Entry validation (12 tests)
`authValidation.test.js`	Auth form validation
`constants.test.js`	Constants and data structure
`store.auth.test.js`	Auth store logic
`backend/tests/auth.utils.test.js`	Backend auth utils
`backend/tests/entries.utils.test.js`	Backend entries utils

VALIDATE.TEST.JS — TEST CASES:

TEST	EXPECTED
`passes for a complete valid entry`	returns [] (no errors)
`error if category is missing`	`validation.categoryRequired`
`error if action is missing`	`validation.actionRequired`
`error if status is missing`	`validation.statusRequired`
`error if description is missing`	`validation.descriptionRequired`
`error if duration is missing`	`validation.durationRequired`
`passes for valid time` `1:30`	returns []
`passes for valid time` `0:05`	returns []
`error for invalid format` `"90"`	`validation.durationFormat`
`error for` `"1:60"` `(minutes out of range)`	`validation.durationFormat`
`returns multiple errors for empty entry`	errors.length ≥ 4
`ValidationError contains error list`	instanceof Error, errors.length === 2

10. PLAYWRIGHT E2E TESTS

CONFIGURATION

Framework	Playwright (TypeScript)
Browser	Chromium (headed)
Base URL	`http://localhost:5173`
Test files	`tests/e2e/*.spec.ts`

RUNNING TESTS

```
# Start dev servers first:
npm run dev

# In a second terminal:
npx playwright test
npx playwright test --ui       # graphical UI
npx playwright test auth       # auth tests only
```

AUTH.SPEC.TS — AUTH AND LOGIN TESTS:

TEST	DESCRIPTION
`shows login form by default`	Email/password inputs visible, submit button present
`demo button logs in as guest`	Click "View demo" → tabs visible, username = "guest"
`wrong credentials show error`	API error visible in `.auth-api-error`
`empty form shows validation errors`	`.field-error` visible with "required" text
`switching to register shows new fields`	Username and confirm password fields appear
`register — validation in Serbian`	Error contains "obavezan"
`logout returns to login page`	Click "Logout" → login form visible

NAVIGATION.SPEC.TS — NAVIGATION AND TABS:

TEST	DESCRIPTION
`all tabs are visible`	Log, Period, Projects, Statistics, Team, Users
`Daily Log tab is active by default`	Class `active`, prev/next buttons and date label visible
`click Period tab shows period view`	"From date", "To date" inputs and shortcut buttons visible
`click Projects tab shows matrix`	Month selector, "Project" and "Total" columns
`click Statistics tab shows cards`	Stat cards "Today", "Total entries" visible
`date navigation — Previous changes date`	Date label differs from initial value
`date navigation — Next returns to today`	Date label returns to original value
`Period shortcut Last 7 days changes dates`	"From date" input has changed

I18N.SPEC.TS — INTERNATIONALIZATION (SR/EN):

TEST	DESCRIPTION
`default language is Serbian`	SRB button has class "active", labels in Serbian

`click ENG switches auth page to English`	ENG button active, "Login", "Email or username" visible
`language saved in localStorage`	`localStorage.getItem('lang')` returns "en"/"sr"
`English persists after reload`	After `page.reload()` English is still active
`validation errors in Serbian`	Error contains "obavezan"
`validation errors in English`	Error contains "required"
`tabs in Serbian by default`	"Dnevni log", "Statistike", "Projekti"
`ENG switches all tabs to English`	"Daily Log", "Statistics", "Projects", "Team"
`logout button is translated`	SR: "Odjava" / EN: "Logout"
`header date is localised`	sr-Latn format ≠ en-GB format
`Projects — months in Serbian`	Select contains "Januar"
`Projects — months in English`	Select contains "January"
`Statistics labels are translated`	SR: "Danas" / EN: "Today"
`Period date labels are translated`	SR: "Od datuma" / EN: "From date"
`ENG button becomes active`	ENG has class "active", SRB does not

11. DATABASE — NEON POSTGRESQL

DETAILS

Service	Neon.tech (free, serverless)
Region	Frankfurt (Azure Germany West)
Project	`qatrackapp`
Branch	`production`
Database	`neondb`

TABLE SCHEMA

```
users
  id, email, username, password_hash
  role: 'user' | 'manager' | 'guest'
  active: boolean
  approved: boolean
  title: text (nullable)
  avatar_data: text (base64, nullable)
  created_at

entries
  id, task_number, category, action
  status, description, duration
  project, date, time, created_at
  user_id → users(id) CASCADE
```

USEFUL SQL QUERIES (NEON CONSOLE):

```
-- Set manager role
UPDATE users SET role = 'manager' WHERE email = 'email@domain.com';

-- Deactivate / activate a user
UPDATE users SET active = false WHERE username = 'user';
UPDATE users SET active = true  WHERE username = 'user';

-- Approve user (approved = can log in)
UPDATE users SET approved = true WHERE email = 'email@domain.com';

-- View all users
SELECT id, email, username, role, active, approved FROM users;
```

12. BACKEND API ROUTES

METHOD + PATH	ACCESS	DESCRIPTION
`POST /api/auth/register`	public	Create user (pending approval)
`POST /api/auth/login`	public	Login (email or username)
`GET /api/auth/me`	JWT	Current user data
`GET /api/entries`	JWT	Entry list (user = own only)
`POST /api/entries`	JWT	New entry (guest cannot)
`PATCH /api/entries/:id`	JWT	Edit entry (guest cannot)
`DELETE /api/entries/:id`	JWT	Delete entry (guest cannot)
`GET /api/stats`	JWT	Statistics for current user
`GET /api/stats/team`	manager	Team report (all users)
`GET /api/users`	manager	All users list
`POST /api/users`	manager	Create user directly
`PATCH /api/users/:id`	manager	Activate / deactivate / role / title
`PATCH /api/users/:id/avatar`	JWT	Upload profile picture (base64 JPEG)
`GET /api/health`	public	Server status — `{ ok: true }`

13. USER ROLES

ROLE	LOGIN	READ ENTRIES	ADD/EDIT/DELETE	TEAM REPORT	USER MANAGEMENT
guest	✅ (demo button)	✅ (all users)	❌	✅	❌
user	✅	✅ (own only)	✅	❌	❌
manager	✅	✅ (all)	✅	✅	✅ (approve, activate, title, role)

> **Manager role** is assigned exclusively via SQL directly in the Neon database. **Guest** access is hardcoded — one guest account exists for demo purposes.

14. SECURITY

MEASURE	IMPLEMENTATION
HTTP security headers	`helmet.js` — CSP, HSTS, X-Frame-Options, X-Content-Type-Options
Brute force protection	`express-rate-limit` — max 10 attempts / 15 min / IP (login + register)
Password hashing	`bcryptjs` — salt rounds: 10
Authentication	JWT token (7 days), payload: `{ userId, role }`
CORS	Only `ALLOWED_ORIGIN=https://qa-daily-log.online`
Payload limit	50 KB maximum request body
DB SSL	`sslmode=require&channel_binding=require` in Neon URL
SQL injection	Parameterised queries everywhere — no string concatenation
DDoS protection	Cloudflare proxy — Pi's real IP is never exposed
Secrets	`.env` in `.gitignore` — never in GitHub

15. TECH STACK

FRONTEND

- React 18 + Vite 5
- Zustand (state management)
- react-i18next (SR/EN bilingual)
- Chart.js (statistics charts)
- SheetJS / xlsx (Excel export)
- Vitest (unit tests)
- Playwright (E2E tests)

BACKEND

- Node.js 20 + Express 4
- pg (PostgreSQL driver)
- bcryptjs (password hashing)
- jsonwebtoken (JWT auth)
- helmet (security headers)
- express-rate-limit
- morgan (HTTP access log)
- dotenv (.env configuration)

INFRASTRUCTURE

- Raspberry Pi 4B (Debian Linux)
- PM2 (process manager)
- cloudflared v2024+ (tunnel daemon)
- Cloudflare Free (proxy + DNS + SSL)
- Namecheap (domain registrar)
- Neon.tech (PostgreSQL cloud)
- GitHub (git repository)

16. LOCAL DEVELOPMENT

STARTING UP

```
# Both servers at once (from root):
npm run dev

# Or separately:
cd backend  && npm start     # port 3001
cd frontend && npm run dev   # port 5173
```

BACKEND/.ENV (LOCAL)

```
DATABASE_URL=postgres://...neon.tech/...
JWT_SECRET=local_secret_key
NODE_ENV=development
ALLOWED_ORIGIN=http://localhost:5173
PORT=3001
```

REPOSITORY STRUCTURE:

```
qa-daily-log/
├── backend/
│   ├── server.js              ← Express (also serves frontend dist/)
│   ├── db.js                  ← PostgreSQL pool + migrations
│   ├── middleware/
│   │   └── auth.js            ← JWT + requireManager
│   ├── routes/
│   │   ├── auth.js, entries.js, stats.js, users.js
│   └── tests/
│       ├── auth.utils.test.js
│       └── entries.utils.test.js
├── frontend/
│   └── src/
│       ├── App.jsx             ← root, tabs, easter eggs, lang switcher
│       ├── store.js            ← Zustand state
│       ├── api.js              ← fetch wrapper
│       ├── constants.js        ← categories, actions, statuses, projects
│       ├── validate.js         ← entry validation
│       ├── i18n.js             ← i18next config (SR/EN)
│       ├── locales/sr.json, en.json
│       ├── sopranos-quotes.js, stranger-things-quotes.js, taxidriver-quotes.js
│       └── components/
│           ├── AuthPage, AddEntry, DailyLog, PeriodView
│           ├── Statistics, ProjectsView, TeamReport
│           ├── UsersPanel, WeeklyReport, EditModal
│           ├── Avatar, Toast, QuoteToast, ErrorBoundary
│           ├── DjokovicWink, FishAnimation, SkierAnimation
│           └── ConfirmDialog
├── tests/e2e/
│   ├── auth.spec.ts           ← auth E2E tests
│   ├── navigation.spec.ts     ← navigation E2E tests
│   └── i18n.spec.ts           ← internationalisation E2E tests
├── playwright.config.ts
└── package.json               ← "dev": concurrently both servers
```

17. TROUBLESHOOTING

PROBLEM	CHECK	SOLUTION
App unreachable	`pm2 status` on Pi	`pm2 restart qa-backend`
App unreachable	`systemctl status cloudflared`	`sudo systemctl restart cloudflared`
Old version displayed	Was rebuild run?	`cd frontend && npm run build`
DB error on startup	Neon console — database status	Check `DATABASE_URL` in `.env`
Login not working (401)	`pm2 logs qa-backend`	Check `JWT_SECRET` in `.env`
CORS error in browser	Network tab in DevTools	Check `ALLOWED_ORIGIN` in `.env`
Pi rebooted — app not working	`pm2 status`, `systemctl status cloudflared`	PM2 and cloudflared are on auto-start (systemd). If not: `pm2 resurrect`

LIVE LOGS ON PI:

```
pm2 logs qa-backend                    # backend logs (live)
pm2 monit                              # PM2 monitor (CPU, RAM, restarts)
sudo journalctl -u cloudflared -f      # Tunnel logs
```


Bosko Sretenovic © 2026